الجمهورية العربية السورية

وزارة التعليم العالي

الجامعة الافتراضية السورية

المعهد التقاني للحاسوب

وظيفة تطبيقية

مقرر: البرمجة والتصميم غرضية التوجه (IPG203)

رمز الوظيفة: TIC_IPG203_F25

نظام إدارة مركز صيانة وإصلاح الأجهزة المنزلية

تقرير الحل: تطبيق مبادئ البرمجة غرضية التوجه بلغة C#

أعضاء الفريق

الاسم	معرّف الطالب	الشعبة
صباح الحيدر	sabah_255155	C1
أحمد العناز	ahmad_234362	C1
ملاز الصعيدي	Malaz_247285	C1
محمد عرول	Moihamad_298199	C1
تالا الطويل	tala_329491	C1

الدكتور (مدرّس المقرر): الدكتور غيث سمير مصطفى

تاريخ التسليم النهائي: 2026-11-12

1. نظرة عامة على النظام

طُوّر النظام بلغة `C#` على شكل تطبيق Console لإدارة مركز صيانة وإصلاح الأجهزة المنزلية. يستقبل المركز ثلاثة أنواع من الأجهزة هي الثلاجة والغسالة والمكيّف، ويسجّل لكل جهاز حالته (`Received` ثم `UnderRepair` ثم `Repaired`) وتكلفة إصلاحه، ويتيح متابعة الأجهزة التي أُصلحت والأجهزة التي ما زالت قيد الصيانة عبر الدالة `GetByStatus` والعدّادات الثابتة في الفئة `Appliance`.

يبيّن الجدول التالي توزيع الفئات على ملفات المشروع:

الملف	العنصر	دوره في النظام
`Interfaces/IRepairable.cs`	واجهة `IRepairable`	العمليات الأساسية المشتركة بين كل الأجهزة
`Models/Appliance.cs`	فئة مجردة `Appliance`	الخصائص المشتركة والدالة المجردة `CalculateRepairCost`
`Models/Refrigerator.cs`	`Refrigerator`	فئة فرعية: الثلاجة
`Models/WashingMachine.cs`	`WashingMachine`	فئة فرعية: الغسالة
`Models/AirConditioner.cs`	`AirConditioner`	فئة فرعية: المكيّف
`Models/RepairStatus.cs`	تعداد `RepairStatus`	حالات الجهاز داخل المركز
`Events/ApplianceStatusChangedEventArgs.cs`	`ApplianceStatusChangedHandler` و`ApplianceStatusChangedEventArgs` و`NotificationKind`	التفويض والحدث وبياناته
`Services/RepairCenter.cs`	`RepairCenter`	الفئة الرئيسية: قائمة الأجهزة والمعالجة متعددة الأشكال
`Utilities/InputValidator.cs`	فئة ثابتة `InputValidator`	التحقق من صحة البيانات المدخلة
`Program.cs`	`Program`	سيناريو تشغيل تجريبي يعرض عمل النظام

لتشغيل المشروع يُفتح الملف `HomeApplianceRepairCenter.sln` في Visual Studio (يُفتح المشروع كاملاً بملفاته) ثم يُضغط `F5` أو `Ctrl+F5`، أو يُنفَّذ الأمر `dotnet run` داخل مجلد المشروع.

2. التجريد (Abstraction)

اعتمد التصميم على مستويين من التجريد: واجهة تصف ما يمكن فعله بأي جهاز، وفئة مجردة تقدّم جزءاً من التنفيذ وتترك الجزء الخاص بكل نوع للفئات الفرعية.

- **الواجهة** `IRepairable` (الملف `Interfaces/IRepairable.cs`): تضم الخاصيتين `SerialNumber` و`Status` والعمليات `CalculateRepairCost()` و`StartRepair()`

و`CompleteRepair()` و`GetDetails()`. هذه العمليات مشتركة بين كل الأجهزة مهما كان نوعها، ولذلك وُضعت في الواجهة.

- **الفئة المجردة** `Appliance` (الملف `Models/Appliance.cs`): تطبّق `IRepairable` وتحتوي على الخصائص المشتركة `SerialNumber` و`Brand` و`CustomerName` و`ReceivedDate` و`DueDate` و`LaborHours` و`PartsCost` و`Status`. وتنفّذ بنفسها `StartRepair()` و`CompleteRepair()` لأن خطواتهما واحدة لكل الأجهزة.
- **الدالة المجردة** `public abstract decimal CalculateRepairCost()`: وهي المرتبطة بحساب تكلفة الإصلاح. لم نكتب لها جسماً في `Appliance` لأن تكلفة إصلاح الثلاجة تختلف عن تكلفة إصلاح الغسالة أو المكيّف، فكل فئة فرعية ملزمة بتقديم حسابها الخاص وإلا لم يُترجم الكود.
- **الخاصية المحمية** `BaseCost`: تجمع ما هو مشترك في الحساب (أجر العمل مضروباً بعدد الساعات مع كلفة القطع) وتستفيد منها الفئات الفرعية.

ونتيجة هذا التصميم لا يمكن إنشاء كائن من `Appliance` مباشرة، وأي كائن في النظام يجب أن يكون ثلاجة أو غسالة أو مكيّفاً.

3. الوراثة وتعدد الأشكال (Inheritance & Polymorphism)

3.1 الوراثة: ثلاث فئات فرعية

ترث الفئات `Refrigerator` و`WashingMachine` و`AirConditioner` من `Appliance`، وتضيف كل واحدة حقولها الخاصة وتعيد تعريف الدالة المجردة بما يناسب طبيعة الجهاز. تُحسب `BaseCost` كما يلي: `LaborHours * 12 + PartsCost` حيث 12 هو أجر الساعة الثابت `LaborRatePerHour`.

الفئة	الحقول الخاصة بها	إعادة تعريف `CalculateRepairCost()`
`Refrigerator`	`_capacityLiters` و`_needsGasRefill`	`BaseCost` مضافاً إليها 35 عند الحاجة إلى تعبئة الغاز، و20 إذا كانت السعة 500 لتر أو أكثر
`WashingMachine`	`_loadCapacityKg` و`_isFrontLoad` و`_drumBearingReplaced`	`BaseCost` مع زيادة 10% للغسالة ذات التحميل الأمامي، ثم إضافة 25 عند استبدال رمان بلي الأسطوانة
`AirConditioner`	`_capacityBtu` و`_isInverter` و`_needsGasRefill` و`_repairedOnSite`	`BaseCost` مضافاً إليها 30 لتعبئة الغاز، و20 لفحص لوحة الإنفرتر، و15 عند الإصلاح في موقع الزبون

وأعادت كل فئة فرعية تعريف `GetDetails()` أيضاً، فهي تستدعي `base.GetDetails()` ثم تضيف بيانات الجهاز الخاصة بها. ومن الأمثلة المحسوبة في البرنامج:

- الثلاجة `RF-100001`: (3 × 12) + 80 = 116، ثم + 35 للغاز، فتكون التكلفة **151.00**.

- الغسالة `WM-200001` (تحميل أمامي): (4 × 12) + 70 = 118، ثم × 1.10 = 129.80، ثم + 25 للبلي، فتكون التكلفة **154.80**.
- المكيّف `AC-300001`: (2 × 12) + 45 = 69، ثم + 30 للغاز + 20 للإنفرتر + 15 للزيارة، فتكون التكلفة **134.00**.

3.2 تعدد الأشكال: معالجة الأجهزة عبر قائمة

تحتوي الفئة الرئيسية `RepairCenter` على القائمة `private readonly List<Appliance> _appliances` التي تضم كائنات مشتقة من أنواع مختلفة. تتعامل الدوال التالية مع الأجهزة كلها بنوع `Appliance` فقط، ويُحدَّد التنفيذ الفعلي وقت التشغيل:

- `CalculateTotalCost()`: حلقة `foreach (Appliance appliance in _appliances)` تستدعي `appliance.CalculateRepairCost()`، فتُنفَّذ نسخة الثلاجة أو الغسالة أو المكيّف بحسب نوع الكائن، من دون أي فحص للنوع بـ `is` أو `if`.
- `CalculateTotalCost(RepairStatus status)` و `GetByStatus(RepairStatus status)`: تحسبان التكلفة الإجمالية للأجهزة في حالة معينة وتعيدان قائمتها.
- `DisplayAllDevices()`: تعرض حالة كل جهاز عبر `GetDetails()` المعاد تعريفها.

في نتيجة التشغيل (القسم 8) كان مجموع تكلفة الأجهزة الستة 735.80، وهو حاصل جمع 151.00 + 230.00 + 154.80 + 44.00 + 134.00 + 22.00 من ثلاثة أنواع مختلفة بحلقة واحدة. وإذا أضفنا نوعاً جديداً من الأجهزة (تلفاز مثلاً) فيكفي أن يرث من `Appliance` دون تعديل `RepairCenter`.

4. التغليف (Encapsulation)

- **الحقول كلها خاصة (`private`)** في جميع الفئات، مثل `_serialNumber` و`_brand` و`_laborHours` و`_partsCost` و`_status` في `Appliance`، و`_capacityLiters` في `Refrigerator`، و`_appliances` في `RepairCenter`. حتى الثوابت مثل `LaborRatePerHour` و`GasRefillFee` معرّفة `private const`.
- **الوصول عبر الخصائص (Properties) مع التحقق**: تمنع الخاصيتان `Brand` و`CustomerName` القيمة الفارغة، وتمنع `LaborHours` و`PartsCost` القيم السالبة، وتشترط `CapacityLiters` و`CapacityBtu` و`LoadCapacityKg` قيماً موجبة.
- **خصائص للقراءة فقط بعد إنشاء الكائن**: `SerialNumber` و`ReceivedDate` و`DueDate` حقول `readonly` تُحدَّد في المُنشئ وخصائصها `get` فقط. وكذلك `Status` التي لا تتغير إلا عبر `StartRepair()` و`CompleteRepair()`، و`IsFrontLoad` و`IsInverter` و`RepairedOnSite` لأنها مواصفات ثابتة للجهاز، و`RepairCenter.Name`.

- **حماية القيم الحساسة**: بعد استدعاء `CompleteRepair()` تستدعي الخصائص `LaborHours` و`PartsCost` و`NeedsGasRefill` و`DrumBearingReplaced` الدالة المحمية `EnsureNotRepaired()` التي ترمي `InvalidOperationException`، فلا تتغير تكلفة جهاز أُصلح بعد احتسابها في الإيراد.
- **حماية القائمة**: تعيد الخاصية `RepairCenter.Appliances` كائناً من النوع `ReadOnlyCollection<Appliance>` فلا يمكن تعديل القائمة من خارج الفئة، وتتم الإضافة فقط عبر `AddAppliance()` التي ترفض الرقم التسلسلي المكرر.

ويظهر أثر ذلك في البند 6 من خرج البرنامج (القسم 8 من هذا التقرير)، حيث رفض النظام تعديل `PartsCost` بعد اكتمال الإصلاح وأي قيمة سالبة.

5. التفويض والأحداث (Events & Delegates)

- **التفويض**: `public delegate void ApplianceStatusChangedHandler(object sender, ApplianceStatusChangedEventArgs e);` في الملف `Events/ApplianceStatusChangedEventArgs.cs`، وهو يحدد شكل الدوال التي يمكنها استقبال إشعار بتغير حالة الجهاز.
- **بيانات الحدث**: الفئة `ApplianceStatusChangedEventArgs` تحمل `SerialNumber` و`OldStatus` و`NewStatus` و`Kind` و`Message`، والتعداد `NotificationKind` قيمه `StatusChanged` و`RepairCompleted` و`DeliveryDelayed`.
- **الحدث**: `public event ApplianceStatusChangedHandler StatusChanged;` في `Appliance`، ويُطلق عبر الدالة `OnStatusChanged(...)` في ثلاث حالات مرتبطة بحالة الصيانة: عند بدء الإصلاح في `StartRepair()` (النوع `StatusChanged`)، وعند اكتماله في `CompleteRepair()` (النوع `RepairCompleted`)، وعند تأخر التسليم في `CheckDelay(DateTime today)` (النوع `DeliveryDelayed`) إذا تجاوز التاريخ موعد التسليم `DueDate` وكان الجهاز غير مُصلَح. ويُرسَل إشعار التأخير مرة واحدة فقط بفضل الحقل `_delayNotified`.
- **الاشتراك**: عند `AddAppliance()` تشترك `RepairCenter` بحدث كل جهاز بالسطر `appliance.StatusChanged += OnApplianceStatusChanged;` ثم تعيد إطلاقه عبر حدثها `ApplianceStatusChanged`. وفي `Program` تشترك الدالة `ShowNotification` بهذا الحدث لتطبع الإشعارات.

دورهما في عمل النظام: لا تعرف الفئة `Appliance` من يستمع إلى أحداثها، فهي تعلن فقط أن شيئاً حدث (بدء إصلاح، اكتمال إصلاح، تأخر تسليم). وبذلك تنفصل منطقة الصيانة عن منطقة التنبيه: يمكن لموظف الاستقبال أن يتلقى إشعاراً بجاهزية الجهاز لإبلاغ الزبون، ولمسؤول المركز أن يتلقى إشعاراً بالأجهزة

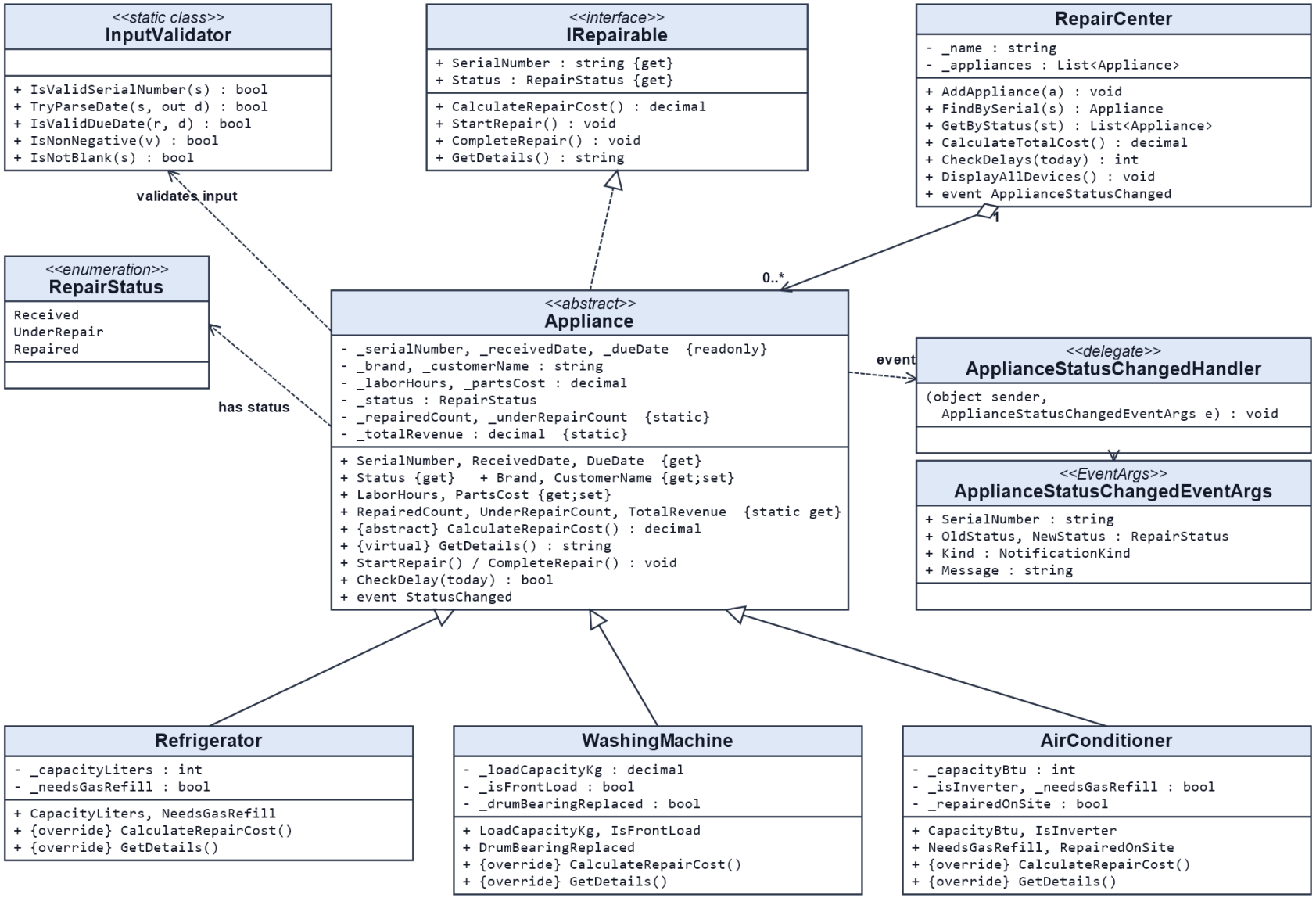
المتأخرة، ويمكن إضافة مشترك جديد (كتابة سجل أو إرسال رسالة) دون تعديل فئات الأجهزة. ويظهر في نتيجة التشغيل إشعار `RepairCompleted` للثلاجة `RF-100001`، وإشعارا `DeliveryDelayed` للجهازين `RF-100002` و`AC-300001` عند الفحص بتاريخ 2026-10-20.

6. الفئات والأعضاء الثابتة (Static Classes & Members)

- **الفئة الثابتة** `InputValidator` (الملف `Utilities/InputValidator.cs`): تحتوي على دوال التحقق `IsValidSerialNumber` (الصيغة حرفان كبيران ثم شرطة ثم ستة أرقام مثل `RF-100001`، بتعبير نمطي `Regex`) و`TryParseDate` (التاريخ بصيغة `yyyy-MM-dd`) و`IsValidDueDate` و`IsNonNegative` و`IsNotBlank`. جُعلت ثابتة لأنها لا تحمل حالة ولا حاجة لإنشاء كائن منها. يستدعيها مُنشئ `Appliance` وخصائصه، ودالة `Date` في `Program`.
- **الخصائص الثابتة في** `Appliance`: `RepairedCount` لعدد الأجهزة التي أُصلحت، و`UnderRepairCount` لعدد الأجهزة قيد الصيانة، و`TotalRevenue` للقيمة الإجمالية لتكاليف الإصلاحات المنجزة. تقرأ كل منها حقلاً ثابتاً خاصاً يُحدَّث داخل `StartRepair()` و`CompleteRepair()`.

في سيناريو التشغيل كانت قيم الخصائص الثابتة بعد التنفيذ: `UnderRepairCount` = 2 و`RepairedCount` = 3 و`TotalRevenue` = 349.80، وهي مجموع 151.00 + 154.80 + 44.00 للأجهزة المصلحة. ويتفق ذلك مع الدالة `RepairCenter.CalculateTotalCost(RepairStatus.Repaired)` التي أعادت القيمة نفسها.

7. رسم UML المبسّط



7.1 شرح العلاقات في الرسم

- **التحقيق (Realization)**: خط متقطع بمثلث مفرغ من `Appliance` إلى `IRepairable`، أي أن الفئة المجردة تطبّق الواجهة.
- **الوراثة (Generalization)**: خط متصل بمثلث مفرغ من `Refrigerator` و`WashingMachine` و`AirConditioner` إلى `Appliance`.
- **التجميع (Aggregation)**: معيّن مفرغ عند `RepairCenter` يدل على أنها تحتوي مجموعة (0..*) من كائنات `Appliance`.
- **الاعتماد (Dependency)**: سهم متقطع من `Appliance` إلى `InputValidator` (يستخدمها للتحقق) وإلى `RepairStatus` (نوع حالة الجهاز) وإلى `ApplianceStatusChangedHandler` (لإطلاق الحدث)، وسهم من التفويض إلى `ApplianceStatusChangedEventArgs` الذي يحمل بياناته.

الرسم مبسّط: عُرضت فيه الحقول والخصائص والدوال الأساسية فقط، وكتبت الخصائص المتجاورة في سطر واحد، والفئات مطابقة للكود الموجود في المشروع.

8. نتيجة تشغيل البرنامج

الخرج التالي ناتج عن ترجمة المشروع وتشغيل `Program.cs` فعلياً. يتضمن السيناريو تسجيل ستة أجهزة (اثنتان من كل نوع)، وسير الصيانة وما يطلقه من أحداث، وفحص التأخر بتاريخ 2026-10-20، والمجاميع، وحالات التحقق والتغليف:

```
=== 1. Registering appliances ===
  Refrigerator | RF-100001 | Samsung | Customer: Ahmad | Status: Received | Due: 2026-10-08 | Cost:
151.00 | 420 L, gas refill: yes
  Refrigerator | RF-100002 | LG | Customer: Salma | Status: Received | Due: 2026-10-12 | Cost: 230.00
| 600 L, gas refill: no
  WashingMachine | WM-200001 | Bosch | Customer: Omar | Status: Received | Due: 2026-10-10 | Cost:
154.80 | 8 kg, front load, drum bearing replaced: yes
  WashingMachine | WM-200002 | Ariston | Customer: Lina | Status: Received | Due: 2026-10-19 | Cost:
44.00 | 6 kg, top load, drum bearing replaced: no
  AirConditioner | AC-300001 | Gree | Customer: Hadi | Status: Received | Due: 2026-10-17 | Cost:
134.00 | 18000 BTU, inverter, gas refill: yes, on-site: yes
  AirConditioner | AC-300002 | Carrier | Customer: Rima | Status: Received | Due: 2026-10-22 | Cost:
22.00 | 12000 BTU, non-inverter, gas refill: no, on-site: no

=== 2. Maintenance workflow (events are raised) ===
  [StatusChanged] Repair started on RF-100001.
  [StatusChanged] Repair started on RF-100002.
  [StatusChanged] Repair started on WM-200001.
  [StatusChanged] Repair started on WM-200002.
  [StatusChanged] Repair started on AC-300001.
  [RepairCompleted] Repair completed on RF-100001. Final cost: 151.00.
  [RepairCompleted] Repair completed on WM-200001. Final cost: 154.80.
  [RepairCompleted] Repair completed on WM-200002. Final cost: 44.00.

=== 3. Checking delayed deliveries on 2026-10-20 ===
  [DeliveryDelayed] Delivery of RF-100002 is late. It was due on 2026-10-12.
  [DeliveryDelayed] Delivery of AC-300001 is late. It was due on 2026-10-17.
Delay events raised: 2
Checking again raises: 0 (each delay is reported once)
```

```
=== 4. Polymorphic processing of all appliances ===
  Refrigerator | RF-100001 | Samsung | Customer: Ahmad | Status: Repaired | Due: 2026-10-08 | Cost:
151.00 | 420 L, gas refill: yes
  Refrigerator | RF-100002 | LG | Customer: Salma | Status: UnderRepair | Due: 2026-10-12 | Cost:
230.00 | 600 L, gas refill: no
  WashingMachine | WM-200001 | Bosch | Customer: Omar | Status: Repaired | Due: 2026-10-10 | Cost:
154.80 | 8 kg, front load, drum bearing replaced: yes
  WashingMachine | WM-200002 | Ariston | Customer: Lina | Status: Repaired | Due: 2026-10-19 | Cost:
44.00 | 6 kg, top load, drum bearing replaced: no
  AirConditioner | AC-300001 | Gree | Customer: Hadi | Status: UnderRepair | Due: 2026-10-17 | Cost:
134.00 | 18000 BTU, inverter, gas refill: yes, on-site: yes
  AirConditioner | AC-300002 | Carrier | Customer: Rima | Status: Received | Due: 2026-10-22 | Cost:
22.00 | 12000 BTU, non-inverter, gas refill: no, on-site: no

=== 5. Totals ===
Total cost of all appliances : 735.80
Cost of repaired appliances  : 349.80
Cost of appliances in repair : 364.00
Cost of waiting appliances   : 22.00
Static counters -> under repair: 2, repaired: 3, revenue: 349.80

=== 6. Validation and encapsulation checks ===
  Invalid serial number -> ArgumentException: Serial number must look like XX-123456.
  Due date before received date -> ArgumentException: Due date cannot be earlier than the received
date.
  Duplicate serial number -> InvalidOperationException: An appliance with serial RF-100001 already
exists.
  Changing parts cost after repair -> InvalidOperationException: The repair of RF-100001 is complete;
its cost data is locked.
  Negative labor hours -> ArgumentOutOfRangeException: Labor hours cannot be negative.
  Completing a repair twice -> InvalidOperationException: Only an appliance under repair can be
completed (RF-100001).
  Completing a repair never started -> InvalidOperationException: Only an appliance under repair can
be completed (AC-300002).
Appliances registered after the failed attempts: 6
```

9. رابط مشروع GitHub

رابط المستودع:

https://github.com/ahmad234362/HomeApplianceRepairCenter

المساهمون المضافون إلى المستودع (الأعضاء الخمسة): صباح الحيدر ، أحمد العناز، ملاز الصعيدي، محمد عرول، تالا الطويل.

ملحق: الشيفرة المصدرية الكاملة للمشروع

الملف: Interfaces/IRepairable.cs

```
using HomeApplianceRepairCenter.Models;

namespace HomeApplianceRepairCenter.Interfaces
{
    /// <summary>
    /// Contract of the core operations shared by every appliance
    /// that is handled by the repair center.
    /// </summary>
    public interface IRepairable
    {
        string SerialNumber { get; }
        RepairStatus Status { get; }

        decimal CalculateRepairCost();
        void StartRepair();
        void CompleteRepair();
        string GetDetails();
    }
}
```

الملف: Models/RepairStatus.cs

```
namespace HomeApplianceRepairCenter.Models
{
    /// <summary>Life-cycle states of an appliance inside the repair center.</summary>
    public enum RepairStatus
    {
        Received,
        UnderRepair,
        Repaired
    }
}
```

الملف: Models/Appliance.cs

```
using System;
using System.Globalization;
using HomeApplianceRepairCenter.Events;
using HomeApplianceRepairCenter.Interfaces;
using HomeApplianceRepairCenter.Utilities;

namespace HomeApplianceRepairCenter.Models
{
    /// <summary>
    /// Abstract base class for every appliance. It implements IRepairable,
    /// keeps the data common to all appliances and leaves the cost
    /// calculation to the concrete appliance types.
    /// </summary>
    public abstract class Appliance : IRepairable
    {
        private const decimal LaborRatePerHour = 12m;

        // Static state shared by all appliances (counts by state and revenue).
        private static int _underRepairCount;
        private static int _repairedCount;
        private static decimal _totalRevenue;

        // Fixed after construction.
        private readonly string _serialNumber;
        private readonly DateTime _receivedDate;
        private readonly DateTime _dueDate;

        // Changeable only through validated members.
        private string _brand;
```

```
        private string _customerName;
        private decimal _laborHours;
        private decimal _partsCost;
        private RepairStatus _status;
        private bool _delayNotified;

        /// <summary>Raised whenever the maintenance state of this appliance changes.</summary>
        public event ApplianceStatusChangedHandler StatusChanged;

        protected Appliance(string serialNumber, string brand, string customerName,
            DateTime receivedDate, DateTime dueDate)
        {
            if (!InputValidator.IsValidSerialNumber(serialNumber))
                throw new ArgumentException("Serial number must look like XX-123456.",
"serialNumber");
            if (!InputValidator.IsValidDueDate(receivedDate, dueDate))
                throw new ArgumentException("Due date cannot be earlier than the received date.",
"dueDate");

            _serialNumber = serialNumber;
            _receivedDate = receivedDate.Date;
            _dueDate = dueDate.Date;
            Brand = brand;
            CustomerName = customerName;
            _status = RepairStatus.Received;
        }

        // ---------- Read-only properties ----------
        public string SerialNumber { get { return _serialNumber; } }
        public DateTime ReceivedDate { get { return _receivedDate; } }
        public DateTime DueDate { get { return _dueDate; } }
        public RepairStatus Status { get { return _status; } }

        // ---------- Properties with validation ----------
        public string Brand
        {
            get { return _brand; }
            set
            {
                if (!InputValidator.IsNotBlank(value))
                    throw new ArgumentException("Brand cannot be empty.");
                _brand = value.Trim();
            }
        }

        public string CustomerName
        {
            get { return _customerName; }
            set
            {
                if (!InputValidator.IsNotBlank(value))
                    throw new ArgumentException("Customer name cannot be empty.");
                _customerName = value.Trim();
            }
        }

        public decimal LaborHours
        {
            get { return _laborHours; }
            set
            {
                EnsureNotRepaired();
                if (!InputValidator.IsNonNegative(value))
                    throw new ArgumentOutOfRangeException("value", "Labor hours cannot be
negative.");
                _laborHours = value;
            }
        }

        public decimal PartsCost
        {
            get { return _partsCost; }
            set
```

```
            {
                EnsureNotRepaired();
                if (!InputValidator.IsNonNegative(value))
                    throw new ArgumentOutOfRangeException("value", "Parts cost cannot be negative.");
                _partsCost = value;
            }
        }

        // ---------- Static properties ----------
        public static int UnderRepairCount { get { return _underRepairCount; } }
        public static int RepairedCount { get { return _repairedCount; } }
        public static decimal TotalRevenue { get { return _totalRevenue; } }

        /// <summary>Labor plus parts; each appliance type adds its own charges on top.</summary>
        protected decimal BaseCost
        {
            get { return _laborHours * LaborRatePerHour + _partsCost; }
        }

        /// <summary>Repair cost, calculated differently by every appliance type.</summary>
        public abstract decimal CalculateRepairCost();

        public virtual string GetDetails()
        {
            return string.Format(CultureInfo.InvariantCulture,
                "{0} | {1} | {2} | Customer: {3} | Status: {4} | Due: {5:yyyy-MM-dd} | Cost: {6:F2}",
                GetType().Name, _serialNumber, _brand, _customerName, _status, _dueDate,
                CalculateRepairCost());
        }

        // ---------- Maintenance workflow ----------
        public void StartRepair()
        {
            if (_status != RepairStatus.Received)
                throw new InvalidOperationException("Only a received appliance can start repair (" +
_serialNumber + ").");

            RepairStatus old = _status;
            _status = RepairStatus.UnderRepair;
            _underRepairCount++;
            OnStatusChanged(NotificationKind.StatusChanged, old,
                "Repair started on " + _serialNumber + ".");
        }

        public void CompleteRepair()
        {
            if (_status != RepairStatus.UnderRepair)
                throw new InvalidOperationException("Only an appliance under repair can be completed
(" + _serialNumber + ").");

            RepairStatus old = _status;
            _status = RepairStatus.Repaired;
            _underRepairCount--;
            _repairedCount++;
            _totalRevenue += CalculateRepairCost();
            OnStatusChanged(NotificationKind.RepairCompleted, old,
                string.Format(CultureInfo.InvariantCulture, "Repair completed on {0}. Final cost:
{1:F2}.", _serialNumber, CalculateRepairCost()));
        }

        public bool IsOverdue(DateTime today)
        {
            return _status != RepairStatus.Repaired && today.Date > _dueDate;
        }

        /// <summary>
        /// Raises a delay notification once, if the appliance passed its due date
        /// without being repaired. Returns true when a notification was raised.
        /// </summary>
        public bool CheckDelay(DateTime today)
        {
            if (!IsOverdue(today) || _delayNotified)
                return false;
```

```
            _delayNotified = true;
            OnStatusChanged(NotificationKind.DeliveryDelayed, _status,
                string.Format(CultureInfo.InvariantCulture, "Delivery of {0} is late. It was due on
{1:yyyy-MM-dd}.", _serialNumber, _dueDate));
            return true;
        }

        /// <summary>Subclasses call this before changing any cost-related data.</summary>
        protected void EnsureNotRepaired()
        {
            if (_status == RepairStatus.Repaired)
                throw new InvalidOperationException("The repair of " + _serialNumber + " is complete;
its cost data is locked.");
        }

        protected virtual void OnStatusChanged(NotificationKind kind, RepairStatus oldStatus, string
message)
        {
            ApplianceStatusChangedHandler handler = StatusChanged;
            if (handler != null)
                handler(this, new ApplianceStatusChangedEventArgs(_serialNumber, oldStatus, _status,
kind, message));
        }
    }
}
```

الملف: Models/Refrigerator.cs

```
using System;
using System.Globalization;

namespace HomeApplianceRepairCenter.Models
{
    /// <summary>A refrigerator: cooling gas and large-capacity handling affect the cost.</summary>
    public class Refrigerator : Appliance
    {
        private const decimal GasRefillFee = 35m;
        private const decimal LargeCapacityFee = 20m;
        private const int LargeCapacityLiters = 500;

        private int _capacityLiters;
        private bool _needsGasRefill;

        public Refrigerator(string serialNumber, string brand, string customerName,
            DateTime receivedDate, DateTime dueDate, int capacityLiters, bool needsGasRefill)
            : base(serialNumber, brand, customerName, receivedDate, dueDate)
        {
            CapacityLiters = capacityLiters;
            _needsGasRefill = needsGasRefill;
        }

        public int CapacityLiters
        {
            get { return _capacityLiters; }
            set
            {
                if (value <= 0)
                    throw new ArgumentOutOfRangeException("value", "Capacity must be positive.");
                _capacityLiters = value;
            }
        }

        public bool NeedsGasRefill
        {
            get { return _needsGasRefill; }
            set
            {
                EnsureNotRepaired();
                _needsGasRefill = value;
            }
        }
```

```
        public override decimal CalculateRepairCost()
        {
            decimal cost = BaseCost;
            if (_needsGasRefill) cost += GasRefillFee;
            if (_capacityLiters >= LargeCapacityLiters) cost += LargeCapacityFee;
            return cost;
        }

        public override string GetDetails()
        {
            return base.GetDetails() + string.Format(" | {0} L, gas refill: {1}",
                _capacityLiters, _needsGasRefill ? "yes" : "no");
        }
    }
}
```

الملف: Models/WashingMachine.cs

```
using System;
using System.Globalization;

namespace HomeApplianceRepairCenter.Models
{
    /// <summary>A washing machine: front-loading models and drum bearing work cost more.</summary>
    public class WashingMachine : Appliance
    {
        private const decimal FrontLoadSurchargeRate = 0.10m;
        private const decimal DrumBearingFee = 25m;

        private decimal _loadCapacityKg;
        private bool _isFrontLoad;
        private bool _drumBearingReplaced;

        public WashingMachine(string serialNumber, string brand, string customerName,
            DateTime receivedDate, DateTime dueDate, decimal loadCapacityKg,
            bool isFrontLoad, bool drumBearingReplaced)
            : base(serialNumber, brand, customerName, receivedDate, dueDate)
        {
            LoadCapacityKg = loadCapacityKg;
            _isFrontLoad = isFrontLoad;
            _drumBearingReplaced = drumBearingReplaced;
        }

        public decimal LoadCapacityKg
        {
            get { return _loadCapacityKg; }
            set
            {
                if (value <= 0m)
                    throw new ArgumentOutOfRangeException("value", "Load capacity must be
positive.");
                _loadCapacityKg = value;
            }
        }

        public bool IsFrontLoad
        {
            get { return _isFrontLoad; }
        }

        public bool DrumBearingReplaced
        {
            get { return _drumBearingReplaced; }
            set
            {
                EnsureNotRepaired();
                _drumBearingReplaced = value;
            }
        }

        public override decimal CalculateRepairCost()
```

```
        {
            decimal cost = BaseCost;
            if (_isFrontLoad) cost += cost * FrontLoadSurchargeRate;
            if (_drumBearingReplaced) cost += DrumBearingFee;
            return cost;
        }

        public override string GetDetails()
        {
            return base.GetDetails() + string.Format(" | {0} kg, {1}, drum bearing replaced: {2}",
                _loadCapacityKg, _isFrontLoad ? "front load" : "top load", _drumBearingReplaced ?
"yes" : "no");
        }
    }
}
```

الملف: Models/AirConditioner.cs

```
using System;
using System.Globalization;

namespace HomeApplianceRepairCenter.Models
{
    /// <summary>An air conditioner: gas refill, inverter board check and on-site visit affect the
cost.</summary>
    public class AirConditioner : Appliance
    {
        private const decimal GasRefillFee = 30m;
        private const decimal InverterCheckFee = 20m;
        private const decimal OnSiteVisitFee = 15m;

        private int _capacityBtu;
        private bool _isInverter;
        private bool _needsGasRefill;
        private bool _repairedOnSite;

        public AirConditioner(string serialNumber, string brand, string customerName,
            DateTime receivedDate, DateTime dueDate, int capacityBtu,
            bool isInverter, bool needsGasRefill, bool repairedOnSite)
            : base(serialNumber, brand, customerName, receivedDate, dueDate)
        {
            CapacityBtu = capacityBtu;
            _isInverter = isInverter;
            _needsGasRefill = needsGasRefill;
            _repairedOnSite = repairedOnSite;
        }

        public int CapacityBtu
        {
            get { return _capacityBtu; }
            set
            {
                if (value <= 0)
                    throw new ArgumentOutOfRangeException("value", "Capacity must be positive.");
                _capacityBtu = value;
            }
        }

        public bool IsInverter
        {
            get { return _isInverter; }
        }

        public bool NeedsGasRefill
        {
            get { return _needsGasRefill; }
            set
            {
                EnsureNotRepaired();
                _needsGasRefill = value;
            }
        }
```

```
        public bool RepairedOnSite
        {
            get { return _repairedOnSite; }
        }

        public override decimal CalculateRepairCost()
        {
            decimal cost = BaseCost;
            if (_needsGasRefill) cost += GasRefillFee;
            if (_isInverter) cost += InverterCheckFee;
            if (_repairedOnSite) cost += OnSiteVisitFee;
            return cost;
        }

        public override string GetDetails()
        {
            return base.GetDetails() + string.Format(" | {0} BTU, {1}, gas refill: {2}, on-site:
{3}",
                _capacityBtu, _isInverter ? "inverter" : "non-inverter",
                _needsGasRefill ? "yes" : "no", _repairedOnSite ? "yes" : "no");
        }
    }
}
```

الملف: Events/ApplianceStatusChangedEventArgs.cs

```
using System;
using HomeApplianceRepairCenter.Models;

namespace HomeApplianceRepairCenter.Events
{
    /// <summary>The kind of notification that the system can raise.</summary>
    public enum NotificationKind
    {
        StatusChanged,
        RepairCompleted,
        DeliveryDelayed
    }

    /// <summary>
    /// Delegate used to notify subscribers that the maintenance state
    /// of an appliance has changed.
    /// </summary>
    public delegate void ApplianceStatusChangedHandler(object sender, ApplianceStatusChangedEventArgs
e);

    /// <summary>Immutable data carried by the status-changed event.</summary>
    public class ApplianceStatusChangedEventArgs : EventArgs
    {
        private readonly string _serialNumber;
        private readonly RepairStatus _oldStatus;
        private readonly RepairStatus _newStatus;
        private readonly NotificationKind _kind;
        private readonly string _message;

        public ApplianceStatusChangedEventArgs(string serialNumber, RepairStatus oldStatus,
            RepairStatus newStatus, NotificationKind kind, string message)
        {
            _serialNumber = serialNumber;
            _oldStatus = oldStatus;
            _newStatus = newStatus;
            _kind = kind;
            _message = message;
        }

        public string SerialNumber { get { return _serialNumber; } }
        public RepairStatus OldStatus { get { return _oldStatus; } }
        public RepairStatus NewStatus { get { return _newStatus; } }
        public NotificationKind Kind { get { return _kind; } }
        public string Message { get { return _message; } }
    }
```

```
}
```

الملف: Services/RepairCenter.cs

```
using System;
using System.Collections.Generic;
using System.Collections.ObjectModel;
using HomeApplianceRepairCenter.Events;
using HomeApplianceRepairCenter.Models;
using HomeApplianceRepairCenter.Utilities;

namespace HomeApplianceRepairCenter.Services
{
    /// <summary>
    /// The main class of the system. It stores the appliances as a list of the
    /// abstract base type and processes all of them polymorphically.
    /// </summary>
    public class RepairCenter
    {
        private readonly string _name;
        private readonly List<Appliance> _appliances = new List<Appliance>();

        /// <summary>Forwards the events of every registered appliance to the outside
world.</summary>
        public event ApplianceStatusChangedHandler ApplianceStatusChanged;

        public RepairCenter(string name)
        {
            if (!InputValidator.IsNotBlank(name))
                throw new ArgumentException("Center name cannot be empty.", "name");
            _name = name.Trim();
        }

        public string Name { get { return _name; } }
        public int Count { get { return _appliances.Count; } }

        public ReadOnlyCollection<Appliance> Appliances
        {
            get { return _appliances.AsReadOnly(); }
        }

        public void AddAppliance(Appliance appliance)
        {
            if (appliance == null)
                throw new ArgumentNullException("appliance");
            if (FindBySerial(appliance.SerialNumber) != null)
                throw new InvalidOperationException("An appliance with serial " +
appliance.SerialNumber + " already exists.");

            _appliances.Add(appliance);
            appliance.StatusChanged += OnApplianceStatusChanged;
        }

        public Appliance FindBySerial(string serialNumber)
        {
            foreach (Appliance appliance in _appliances)
            {
                if (appliance.SerialNumber == serialNumber)
                    return appliance;
            }
            return null;
        }

        public List<Appliance> GetByStatus(RepairStatus status)
        {
            List<Appliance> result = new List<Appliance>();
            foreach (Appliance appliance in _appliances)
            {
                if (appliance.Status == status)
                    result.Add(appliance);
            }
            return result;
```

```
        }

        /// <summary>Sums the cost of all appliances; each call to CalculateRepairCost is
polymorphic.</summary>
        public decimal CalculateTotalCost()
        {
            decimal total = 0m;
            foreach (Appliance appliance in _appliances)
                total += appliance.CalculateRepairCost();
            return total;
        }

        public decimal CalculateTotalCost(RepairStatus status)
        {
            decimal total = 0m;
            foreach (Appliance appliance in GetByStatus(status))
                total += appliance.CalculateRepairCost();
            return total;
        }

        public void DisplayAllDevices()
        {
            foreach (Appliance appliance in _appliances)
                Console.WriteLine("  " + appliance.GetDetails());
        }

        /// <summary>Checks every appliance against today's date and returns how many delay events
were raised.</summary>
        public int CheckDelays(DateTime today)
        {
            int raised = 0;
            foreach (Appliance appliance in _appliances)
            {
                if (appliance.CheckDelay(today))
                    raised++;
            }
            return raised;
        }

        private void OnApplianceStatusChanged(object sender, ApplianceStatusChangedEventArgs e)
        {
            ApplianceStatusChangedHandler handler = ApplianceStatusChanged;
            if (handler != null)
                handler(sender, e);
        }
    }
}
```

الملف: Utilities/InputValidator.cs

```
using System;
using System.Globalization;
using System.Text.RegularExpressions;

namespace HomeApplianceRepairCenter.Utilities
{
    /// <summary>
    /// Static helper class that validates the data entered for appliances.
    /// It holds no state, so it is declared static and is never instantiated.
    /// </summary>
    public static class InputValidator
    {
        private const string DateFormat = "yyyy-MM-dd";

        // Two capital letters, a dash, then six digits (e.g. RF-100001).
        private static readonly Regex SerialPattern = new Regex(@"^[A-Z]{2}-\d{6}$");

        public static bool IsNotBlank(string text)
        {
            return !string.IsNullOrWhiteSpace(text);
        }
```

```
        public static bool IsValidSerialNumber(string serialNumber)
        {
            return serialNumber != null && SerialPattern.IsMatch(serialNumber);
        }

        public static bool TryParseDate(string text, out DateTime date)
        {
            return DateTime.TryParseExact(text, DateFormat, CultureInfo.InvariantCulture,
                DateTimeStyles.None, out date);
        }

        public static bool IsValidDueDate(DateTime receivedDate, DateTime dueDate)
        {
            return dueDate.Date >= receivedDate.Date;
        }

        public static bool IsNonNegative(decimal value)
        {
            return value >= 0m;
        }
    }
}
```

الملف: Program.cs

```
using System;
using HomeApplianceRepairCenter.Events;
using HomeApplianceRepairCenter.Models;
using HomeApplianceRepairCenter.Services;
using HomeApplianceRepairCenter.Utilities;

namespace HomeApplianceRepairCenter
{
    internal static class Program
    {
        private static void Main()
        {
            RepairCenter center = new RepairCenter("Damascus Home Appliance Repair Center");
            center.ApplianceStatusChanged += ShowNotification;

            Console.WriteLine("=== 1. Registering appliances ===");
            center.AddAppliance(new Refrigerator("RF-100001", "Samsung", "Ahmad", Date("2026-10-01"),
Date("2026-10-08"), 420, true));
            center.AddAppliance(new Refrigerator("RF-100002", "LG", "Salma", Date("2026-10-05"),
Date("2026-10-12"), 600, false));
            center.AddAppliance(new WashingMachine("WM-200001", "Bosch", "Omar", Date("2026-10-03"),
Date("2026-10-10"), 8m, true, true));
            center.AddAppliance(new WashingMachine("WM-200002", "Ariston", "Lina", Date("2026-10-
12"), Date("2026-10-19"), 6m, false, false));
            center.AddAppliance(new AirConditioner("AC-300001", "Gree", "Hadi", Date("2026-10-10"),
Date("2026-10-17"), 18000, true, true, true));
            center.AddAppliance(new AirConditioner("AC-300002", "Carrier", "Rima", Date("2026-10-
15"), Date("2026-10-22"), 12000, false, false, false));

            // Labor hours and parts cost are entered after the diagnosis.
            SetWork(center, "RF-100001", 3m, 80m);
            SetWork(center, "RF-100002", 5m, 150m);
            SetWork(center, "WM-200001", 4m, 70m);
            SetWork(center, "WM-200002", 2m, 20m);
            SetWork(center, "AC-300001", 2m, 45m);
            SetWork(center, "AC-300002", 1m, 10m);
            center.DisplayAllDevices();

            Console.WriteLine();
            Console.WriteLine("=== 2. Maintenance workflow (events are raised) ===");
            center.FindBySerial("RF-100001").StartRepair();
            center.FindBySerial("RF-100002").StartRepair();
            center.FindBySerial("WM-200001").StartRepair();
            center.FindBySerial("WM-200002").StartRepair();
            center.FindBySerial("AC-300001").StartRepair();
            center.FindBySerial("RF-100001").CompleteRepair();
            center.FindBySerial("WM-200001").CompleteRepair();
```

```
            center.FindBySerial("WM-200002").CompleteRepair();

            Console.WriteLine();
            Console.WriteLine("=== 3. Checking delayed deliveries on 2026-10-20 ===");
            int delayed = center.CheckDelays(Date("2026-10-20"));
            Console.WriteLine("Delay events raised: " + delayed);
            Console.WriteLine("Checking again raises: " + center.CheckDelays(Date("2026-10-20")) + "
(each delay is reported once)");

            Console.WriteLine();
            Console.WriteLine("=== 4. Polymorphic processing of all appliances ===");
            center.DisplayAllDevices();

            Console.WriteLine();
            Console.WriteLine("=== 5. Totals ===");
            Console.WriteLine("Total cost of all appliances : {0:F2}", center.CalculateTotalCost());
            Console.WriteLine("Cost of repaired appliances  : {0:F2}",
center.CalculateTotalCost(RepairStatus.Repaired));
            Console.WriteLine("Cost of appliances in repair : {0:F2}",
center.CalculateTotalCost(RepairStatus.UnderRepair));
            Console.WriteLine("Cost of waiting appliances   : {0:F2}",
center.CalculateTotalCost(RepairStatus.Received));
            Console.WriteLine("Static counters -> under repair: {0}, repaired: {1}, revenue: {2:F2}",
                Appliance.UnderRepairCount, Appliance.RepairedCount, Appliance.TotalRevenue);

            Console.WriteLine();
            Console.WriteLine("=== 6. Validation and encapsulation checks ===");
            Try("Invalid serial number", delegate { new Refrigerator("XX123", "LG", "Test",
Date("2026-10-01"), Date("2026-10-02"), 300, false); });
            Try("Due date before received date", delegate { new Refrigerator("RF-100009", "LG",
"Test", Date("2026-10-05"), Date("2026-10-01"), 300, false); });
            Try("Duplicate serial number", delegate { center.AddAppliance(new Refrigerator("RF-
100001", "LG", "Test", Date("2026-10-01"), Date("2026-10-02"), 300, false)); });
            Try("Changing parts cost after repair", delegate { center.FindBySerial("RF-
100001").PartsCost = 10m; });
            Try("Negative labor hours", delegate { center.FindBySerial("RF-100002").LaborHours = -1m;
});
            Try("Completing a repair twice", delegate { center.FindBySerial("RF-
100001").CompleteRepair(); });
            Try("Completing a repair never started", delegate { center.FindBySerial("AC-
300002").CompleteRepair(); });
            Console.WriteLine("Appliances registered after the failed attempts: " + center.Count);
        }

        private static void ShowNotification(object sender, ApplianceStatusChangedEventArgs e)
        {
            Console.WriteLine("  [{0}] {1}", e.Kind, e.Message);
        }

        private static DateTime Date(string text)
        {
            DateTime result;
            if (!InputValidator.TryParseDate(text, out result))
                throw new FormatException("Invalid date: " + text);
            return result;
        }

        private static void SetWork(RepairCenter center, string serial, decimal hours, decimal
partsCost)
        {
            Appliance appliance = center.FindBySerial(serial);
            appliance.LaborHours = hours;
            appliance.PartsCost = partsCost;
        }

        private static void Try(string title, Action action)
        {
            try
            {
                action();
                Console.WriteLine("  {0}: no error (unexpected)", title);
            }
            catch (Exception ex)
```

```
            {
                string message = ex.Message;
                int newLine = message.IndexOf('\n');
                if (newLine >= 0) message = message.Substring(0, newLine).TrimEnd();
                Console.WriteLine("  {0} -> {1}: {2}", title, ex.GetType().Name, message);
            }
        }
    }
}
```

الملف: HomeApplianceRepairCenter.csproj

```
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net8.0</TargetFramework>
    <RootNamespace>HomeApplianceRepairCenter</RootNamespace>
    <Nullable>disable</Nullable>
    <ImplicitUsings>disable</ImplicitUsings>
  </PropertyGroup>

</Project>
```